МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КРЕМЕНЧУЦЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ МИХАЙЛА ОСТРОГРАДСЬКОГО
НАВЧАЛЬНО-НАУКОВИЙ ІНСТИТУТ ЕЛЕКТРИЧНОЇ ІНЖЕНЕРІЇ
ТА ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ

Кафедра автоматизації та інформаційних систем

ОСВІТНІЙ КОМПОНЕНТ
«ОРГАНІЗАЦІЯ БАЗ ДАНИХ»

ЗВІТ
З ЛАБОРАТОРНОЇ РОБОТИ № 7

Виконав:
студент групи КН-25-1
Рафієв Б. С.
Перевірив:
асистент кафедри АІС
Андреєв П.І.

Кременчук 2026

Тема: **Збережені процедури.**

Мета: навчитися створювати збережені процедури.

Порядок виконання роботи

Лабораторну роботу виконано в середовищі Microsoft SQL Server Management Studio 22. Для першої частини використано навчальну базу даних Sales111, створену під час лабораторної роботи № 2; для індивідуального завдання – базу даних LibraryDB (варіант № 1 «Бібліотека»). Збережені процедури створено командою CREATE PROCEDURE; у процедурах використано вхідні та вихідні параметри, змінні, конструкції IF…ELSE, BEGIN…END, RAISERROR і RETURN. Результати виконання наведено на рисунках. Після зняття скріншотів змінені під час демонстрації дані відновлено до початкового стану.

1. Збережені процедури для бази даних Sales111

У цій частині створено вісім збережених процедур для бази даних Sales111. Кожна процедура розв’язує окреме завдання з методичних вказівок: змінює дату замовлень, видаляє замовлення заданого замовника, збільшує бонуси, перепризначає замовлення між продавцями, видаляє замовників і продавців без замовлень, збільшує комісійні та знаходить найдовше прізвище за заданою літерою. Для кожного прикладу наведено код створення процедури та результат її виконання.

1.1. Створити процедуру для зміни дати оформлення замовлень за 10.03.2024 на 11.03.2024 для всіх рядків таблиці Orders.

```
CREATE PROCEDURE ChangeOrderDate
AS
BEGIN
    UPDATE Orders
    SET DATE_OD = '2024-03-11'
    WHERE DATE_OD = '2024-03-10';
END;
GO

EXEC ChangeOrderDate;

SELECT ID_OD, DATE_OD
FROM Orders
WHERE ID_OD = 3012;
```

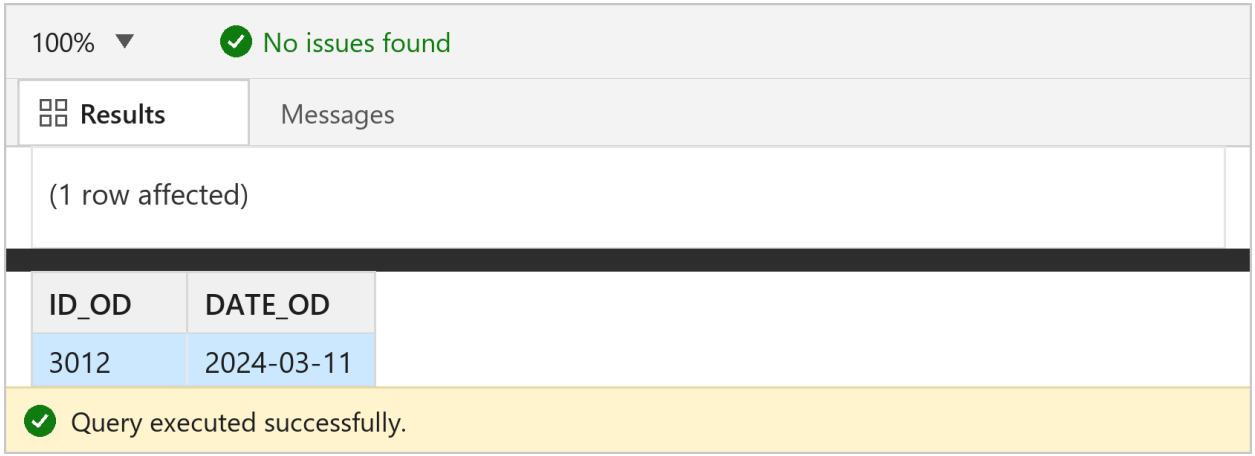


Рисунок 1 – Результат виконання процедури 1.1

У таблиці Orders не було замовлень за 10.03.2024, тому для перевірки процедури додано тестовий рядок 3012 (замовник 2006, продавець 1001). Після виконання процедури дата змінилася на 11.03.2024; після зняття скріншота рядок видалено.

1.2. Створити процедуру для видалення всіх замовлень замовника Клименко з таблиці Orders.

```
CREATE PROCEDURE DeleteCustomerOrders
    @customer NVARCHAR(50) = N'Клименко'
AS
BEGIN
    DELETE FROM Orders
    WHERE ID_CM = (SELECT ID_CM FROM Customers
                   WHERE NAME_CM = @customer);
END;
GO

EXEC DeleteCustomerOrders @customer = N'Клименко';

SELECT COUNT(*) AS Залишилося_замовлень
FROM Orders
WHERE ID_CM = 2006;
```

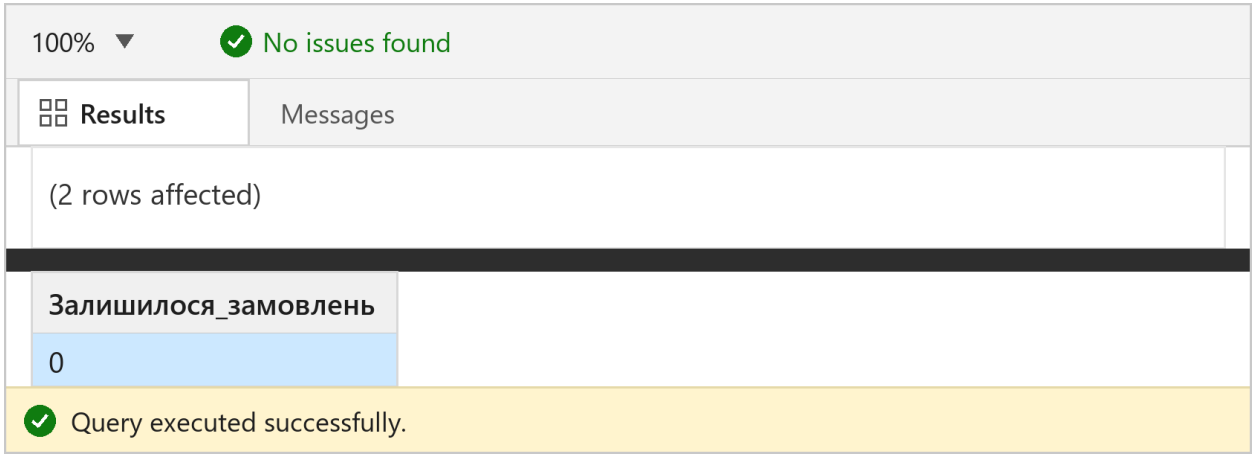


Рисунок 2 – Результат виконання процедури 1.2

Процедура приймає прізвище замовника; для Клименка використано значення за замовчуванням. Видалено замовлення 3008 і 3011; після перевірки їх відновлено.

1.3. Створити процедуру для збільшення бонусного балу всіх замовників у Києві на 100.

```
CREATE PROCEDURE IncreaseKyivBonus
    @amount INT = 100
AS
BEGIN
    UPDATE Customers
    SET BONUS_CM = BONUS_CM + @amount
    WHERE CITY_CM = N'Київ';
END;
GO

EXEC IncreaseKyivBonus @amount = 100;

SELECT ID_CM, NAME_CM, CITY_CM, BONUS_CM
FROM Customers
WHERE CITY_CM = N'Київ'
ORDER BY ID_CM;
```

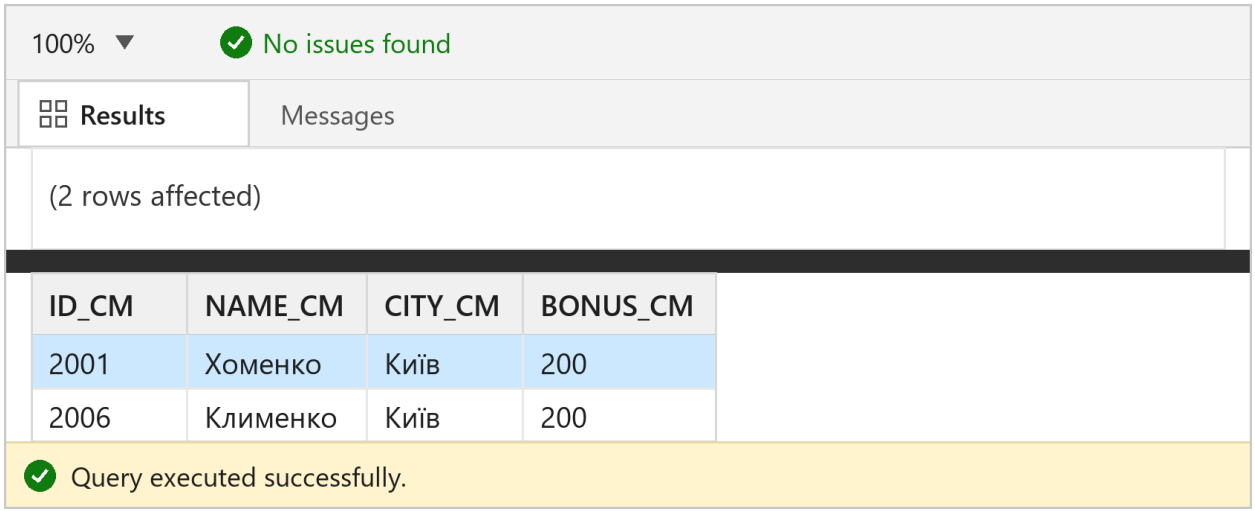


Рисунок 3 – Результат виконання процедури 1.3

У Києві проживають замовники 2001 Хоменко і 2006 Клименко; їхній бонус збільшився зі 100 до 200. Після перевірки початкові значення відновлено.

1.4. Створити процедуру для перепризначення замовлень одного продавця (Чепуренко) іншому продавцеві (Маренич).

```
CREATE PROCEDURE ReassignSellerOrders
    @from NVARCHAR(50),
    @to NVARCHAR(50)
AS
BEGIN
    DECLARE @from_id INT, @to_id INT;

    SELECT @from_id = ID_SP FROM Salespeople WHERE NAME_SP = @from;
    SELECT @to_id = ID_SP FROM Salespeople WHERE NAME_SP = @to;

    IF @from_id IS NULL OR @to_id IS NULL
    BEGIN
        RAISERROR(N'Продавця не знайдено', 16, 1);
        RETURN;
    END;

    UPDATE Orders
    SET ID_SP = @to_id
    WHERE ID_SP = @from_id;
END;
GO

EXEC ReassignSellerOrders @from = N'Чепуренко', @to = N'Маренич';

SELECT o.ID_OD, c.NAME_CM AS Замовник,
       s.NAME_SP AS Продавець, o.AMOUNT_OD AS Сума
FROM Orders o
JOIN Customers c ON c.ID_CM = o.ID_CM
JOIN Salespeople s ON s.ID_SP = o.ID_SP
WHERE o.ID_OD IN (3002, 3005, 3007, 3010)
ORDER BY o.ID_OD;
```

100% ▼ ✓ No issues found

Results | Messages

(3 rows affected)

ID_OD	Замовник	Продавець	Сума
3002	Петренко	Маренич	1900.10
3005	Луценко	Маренич	5160.45
3007	Гора	Маренич	375.75
3010	Гора	Маренич	1309.95

✓ Query executed successfully.

Рисунок 4 – Результат виконання процедури 1.4

Замовлення 3005, 3007 і 3010 передано від Чепуренка до Маренича; замовлення 3002 вже належало Мареничу й залишилося без змін. Після перевірки приналежність відновлено.

1.5. Створити процедуру для видалення всіх замовників, що не мають поточних замовлень.

```
CREATE PROCEDURE DeleteCustomersWithoutOrders
AS
BEGIN
    DELETE FROM Customers
    WHERE NOT EXISTS (
        SELECT 1 FROM Orders o
        WHERE o.ID_CM = Customers.ID_CM
    );
END;
GO

EXEC DeleteCustomersWithoutOrders;

SELECT COUNT(*) AS Залишилося
FROM Customers
WHERE ID_CM = 2005;
```

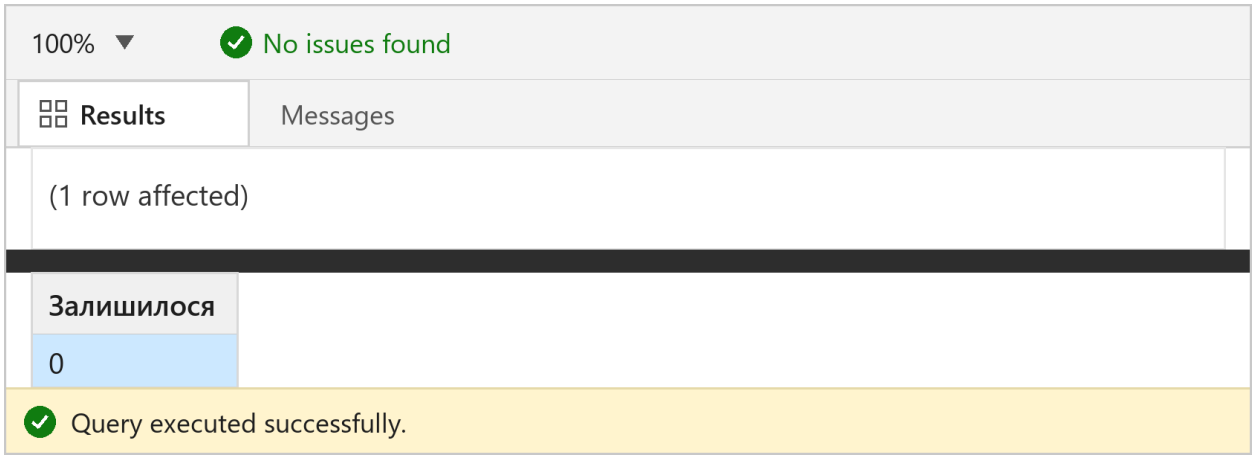


Рисунок 5 – Результат виконання процедури 1.5

Єдиним замовником без замовлень був 2005 Шевченко; процедура видалила його. Після перевірки запис відновлено.

1.6. Створити процедуру для видалення всіх продавців, що не мають поточних замовлень.

```
CREATE PROCEDURE DeleteSellersWithoutOrders
AS
BEGIN
    DELETE FROM Salespeople
    WHERE NOT EXISTS (
        SELECT 1 FROM Orders o
        WHERE o.ID_SP = Salespeople.ID_SP
    );
END;
GO

EXEC DeleteSellersWithoutOrders;

SELECT COUNT(*) AS Залишилося
FROM Salespeople
WHERE ID_SP = 1005;
```

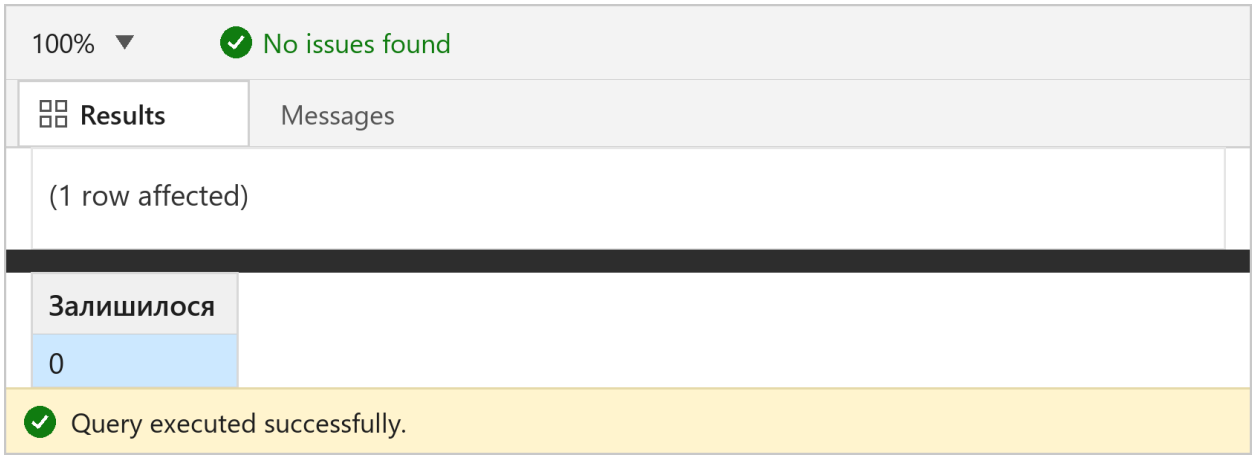


Рисунок 6 – Результат виконання процедури 1.6

Єдиним продавцем без замовлень був 1005 Ковальчук; процедура видалила його. Після перевірки запис відновлено.

1.7. Створити процедуру для збільшення на 20% комісійних продавців, сума поточних замовлень яких перевищує 3000.

```
CREATE PROCEDURE IncreaseTopSellerCommission
    @min_total MONEY = 3000,
    @percent DECIMAL(5, 2) = 20
AS
BEGIN
    UPDATE Salespeople
    SET COMM_SP = COMM_SP * (1 + @percent / 100)
    WHERE ID_SP IN (
        SELECT ID_SP
        FROM Orders
        GROUP BY ID_SP
        HAVING SUM(AMOUNT_OD) > @min_total
    );
END;
GO

EXEC IncreaseTopSellerCommission @min_total = 3000, @percent = 20;

SELECT s.ID_SP, s.NAME_SP,
       SUM(o.AMOUNT_OD) AS Сума_замовлень,
       s.COMM_SP
FROM Salespeople s
JOIN Orders o ON o.ID_SP = s.ID_SP
GROUP BY s.ID_SP, s.NAME_SP, s.COMM_SP
HAVING SUM(o.AMOUNT_OD) > 3000
ORDER BY s.ID_SP;
```

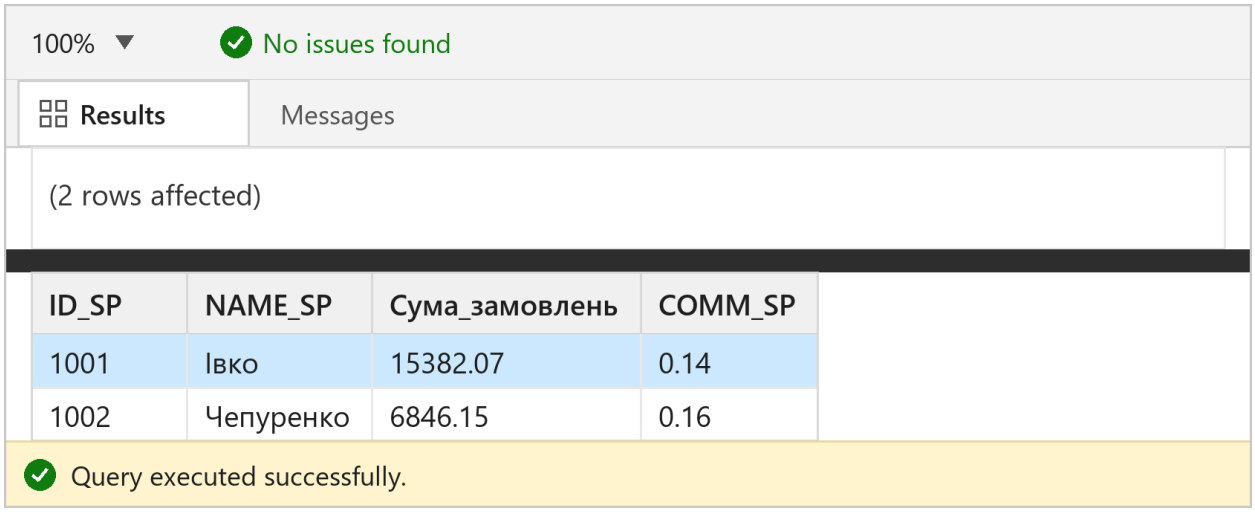


Рисунок 7 – Результат виконання процедури 1.7

Поріг 3000 перевищують продавці 1001 Івко (15382,07) і 1002 Чепуренко (6846,15). Після збільшення на 20% комісійні становлять 0,14 і 0,16 (округлення до двох знаків). Після перевірки значення відновлено.

1.8. Створити процедуру для визначення найдовшого прізвища замовника, що починається із вказаної літери.

```
CREATE PROCEDURE LongestCustomerName
    @letter NCHAR(1)
AS
BEGIN
    IF NOT EXISTS (
        SELECT 1 FROM Customers WHERE NAME_CM LIKE @letter + N'%'
    )
    BEGIN
        RAISERROR(N'Замовників із такої літери не знайдено', 16, 1);
        RETURN;
    END;

    SELECT TOP 1 NAME_CM AS Найдовше_прізвище
    FROM Customers
    WHERE NAME_CM LIKE @letter + N'%'
    ORDER BY LEN(NAME_CM) DESC, NAME_CM;
END;
GO

EXEC LongestCustomerName @letter = N'K';
```



Рисунок 8 – Результат виконання процедури 1.8

Для літери К найдовшим є прізвище Клименко (8 літер). Для літери Г процедура повернула б Гірченко (8 літер), а не Гора (4).

2. Індивідуальне завдання. База даних LibraryDB

Індивідуальне завдання виконано для варіанта № 1 – предметна область «Бібліотека». Використано базу даних LibraryDB із таблицями Author, Publisher, Book, Reader і BookLoan. Створено три збережені процедури: AddBook (додавання книги з перевіркою вхідних даних), ReturnBook (повернення книги за номером видачі) і ReaderLoanReport (статистика видач читача через вихідні параметри OUTPUT).

2.1. Створити процедуру AddBook для додавання нової книги з перевіркою вхідних даних.

```
CREATE PROCEDURE AddBook
    @title NVARCHAR(100),
    @author INT,
    @publisher INT,
    @year INT,
    @price MONEY
AS
BEGIN
    IF @price <= 0
    BEGIN
        RAISERROR(N'Ціна книги повинна бути додатною', 16, 1);
        RETURN;
    END;

    IF NOT EXISTS (SELECT 1 FROM Author WHERE id_author = @author)
    BEGIN
        RAISERROR(N'Автора з таким кодом не існує', 16, 1);
        RETURN;
    END;

    IF NOT EXISTS (SELECT 1 FROM Publisher WHERE id_publisher = @publisher)
    BEGIN
        RAISERROR(N'Видавництва з таким кодом не існує', 16, 1);
        RETURN;
    END;

    INSERT INTO Book (title, id_author, id_publisher, year_pub, price)
    VALUES (@title, @author, @publisher, @year, @price);
END;
GO

EXEC AddBook @title = N'Тіні забутих предків', @author = 2,
             @publisher = 2, @year = 2023, @price = 300;

SELECT * FROM Book WHERE title = N'Тіні забутих предків';
```

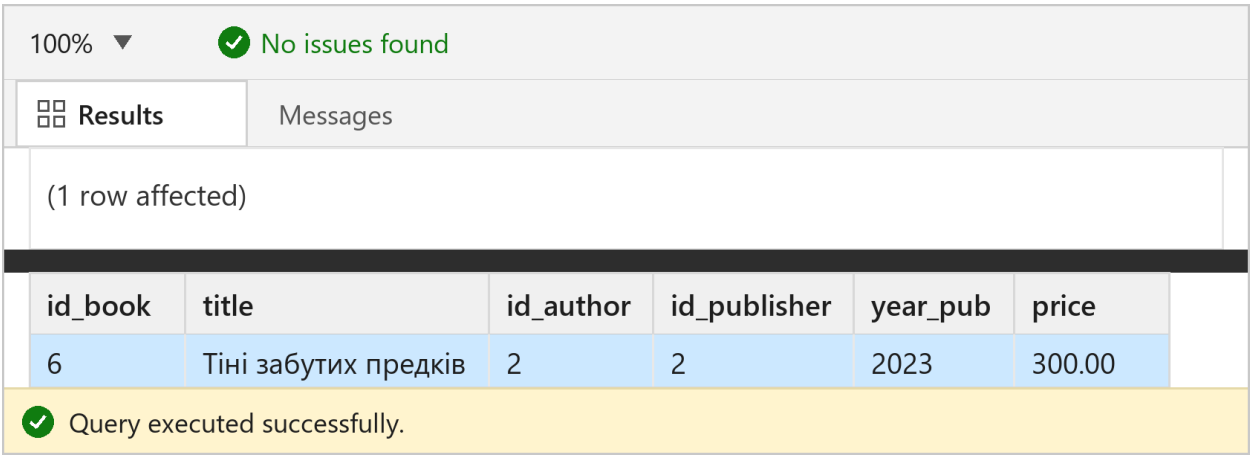


Рисунок 9 – Результат виконання процедури 2.1

Процедура перевіряє додатність ціни та наявність автора й видавництва. Нова книга отримала код 6; після перевірки її видалено, а лічильник ідентичності повернуто до значення 5.

2.2. Створити процедуру ReturnBook для повернення книги за номером видачі.

```
CREATE PROCEDURE ReturnBook
    @loan INT,
    @date DATE = NULL
AS
BEGIN
    IF NOT EXISTS (SELECT 1 FROM BookLoan WHERE id_loan = @loan)
    BEGIN
        RAISERROR(N'Видачу з таким номером не знайдено', 16, 1);
        RETURN;
    END;

    IF EXISTS (SELECT 1 FROM BookLoan
               WHERE id_loan = @loan AND return_date IS NOT NULL)
    BEGIN
        RAISERROR(N'Книга за цією видачею вже повернута', 16, 1);
        RETURN;
    END;

    IF @date IS NULL
        SET @date = CAST(GETDATE() AS DATE);

    UPDATE BookLoan
    SET return_date = @date
    WHERE id_loan = @loan;
END;
GO

EXEC ReturnBook @loan = 2, @date = '2026-09-30';

SELECT * FROM BookLoan WHERE id_loan = 2;
```

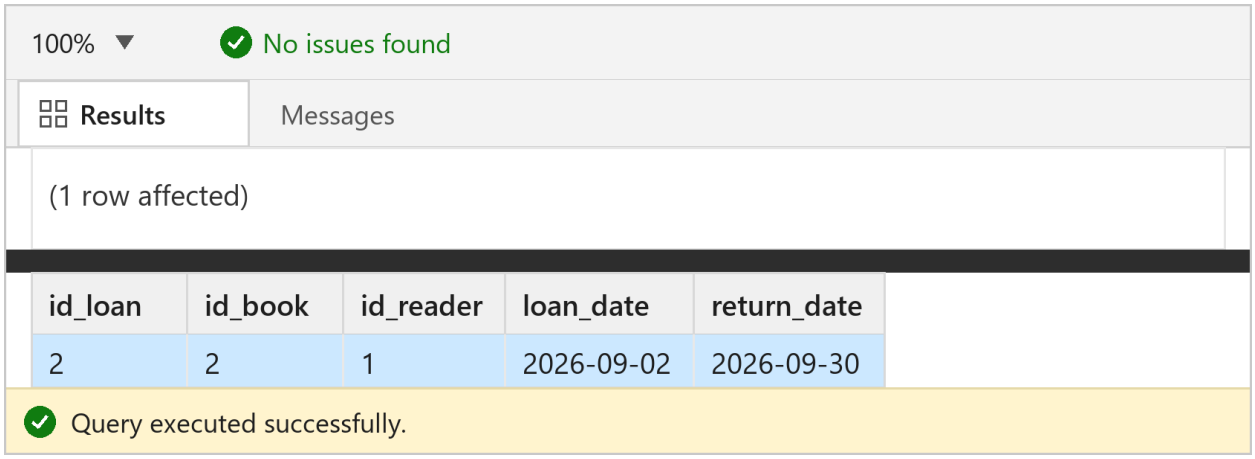


Рисунок 10 – Результат виконання процедури 2.2

Видачу № 2 повернуто 30.09.2026; якщо дату не вказати, процедура використовує поточну дату (GETDATE). Після перевірки дату повернення знову встановлено в NULL.

2.3. Створити процедуру ReaderLoanReport, яка повертає кількість видач і повернутих книг читача через вихідні параметри.

```
CREATE PROCEDURE ReaderLoanReport
    @reader INT,
    @total INT OUTPUT,
    @returned INT OUTPUT
AS
BEGIN
    IF NOT EXISTS (SELECT 1 FROM Reader WHERE id_reader = @reader)
    BEGIN
        RAISERROR(N'Читача з таким кодом не знайдено', 16, 1);
        RETURN;
    END;

    SELECT @total = COUNT(*),
           @returned = SUM(CASE WHEN return_date IS NOT NULL THEN 1 ELSE 0 END)
    FROM BookLoan
    WHERE id_reader = @reader;
END;
GO

DECLARE @total INT, @returned INT;
EXEC ReaderLoanReport @reader = 1,
     @total = @total OUTPUT, @returned = @returned OUTPUT;
SELECT @total AS Усього_видач, @returned AS Повернуто_книг;
```

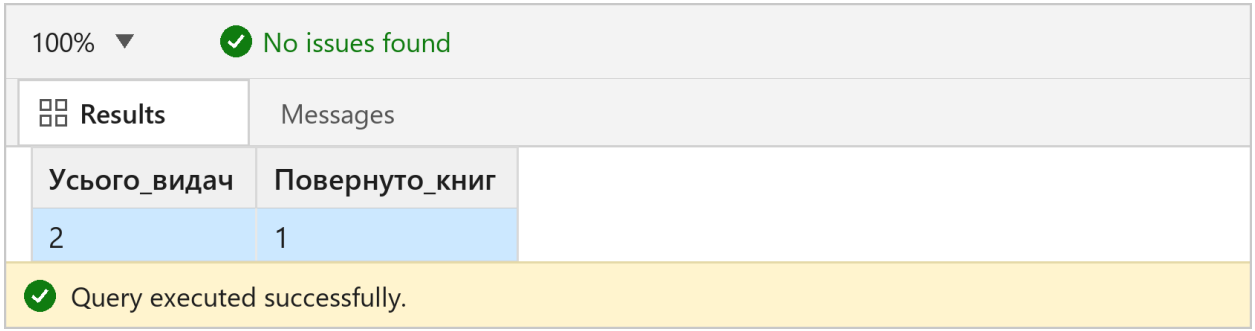


Рисунок 11 – Результат виконання процедури 2.3

Читач 1 має дві видачі (№ 1 і № 2), з них одна повернута. Процедура повертає ці показники через вихідні параметри OUTPUT.

Відповіді на контрольні питання

1. Що являють собою збережені процедури?

Збережена процедура (stored procedure) – це іменований набір команд Transact-SQL, який зберігається в базі даних у вигляді об’єкта й виконується на сервері за командою EXEC. Процедура може містити один або кілька операторів SQL, оголошення змінних, керуючі конструкції, виклики інших процедур і функцій. Під час першого виконання сервер виконує синтаксичний аналіз, оптимізацію та компіляцію, після чого зберігає план виконання в кеші.

На відміну від звичайного скрипта, процедура є постійним об’єктом бази даних: вона має ім’я, власника, параметри та зберігається разом з іншими об’єктами. Виклик процедури виглядає як EXEC dbo.ProcedureName; або EXEC ProcedureName @param = value;. Результатом роботи можуть бути один або кілька наборів рядків (SELECT), вихідні параметри OUTPUT, код повернення RETURN і повідомлення PRINT або RAISERROR.

Приклад найпростішої процедури, створеної в цій роботі:

```
CREATE PROCEDURE DeleteSellersWithoutOrders
AS
BEGIN
    DELETE FROM Salespeople
    WHERE NOT EXISTS (
        SELECT 1 FROM Orders o
        WHERE o.ID_SP = Salespeople.ID_SP
    );
END;
GO

EXEC DeleteSellersWithoutOrders;
```

Процедура зберігається в базі даних Sales111 і після створення може викликатися з будь-якого клієнтського застосунку, який має право на її виконання.

2. Які основні переваги збережених процедур?

1. Продуктивність. Сервер компілює процедуру один раз і повторно використовує план виконання, тому накладні витрати на розбір і оптимізацію запиту зменшуються. Крім того, замість багатьох окремих операторів клієнт передає лише виклик процедури, що скорочує мережевий трафік.

2. Модульність і повторне використання. Один раз описана логіка застосовується в різних застосунках і звітах, її не потрібно дублювати в коді клієнта.

3. Безпека. Можна надати користувачу право EXECUTE на процедуру, не даючи прямого доступу до таблиць. Процедура виконується з правами власника (у межах ланцюжка володіння), тому користувач бачить лише дозволені дані.

4. Централізація бізнес-логіки. Правила обробки даних зберігаються на сервері й застосовуються однаково для всіх клієнтів; зміни вносять в одному місці.

5. Захист від SQL-ін’єкцій. Параметри передаються окремо від тексту запиту, тому значення не інтерпретуються як фрагменти SQL.

6. Зручність супроводу. Процедуру можна змінити командою ALTER PROCEDURE, не перекомпілюючи клієнтські програми.

7. Керування транзакціями. У процедурі можна відкривати, фіксувати й відкочувати транзакції, обробляти помилки через TRY…CATCH і повертати узгоджений результат.

8. Повторне використання планів і зменшення навантаження на клієнт: складні обчислення виконуються на сервері, а клієнт отримує готовий результат.

3. Як створити нову збережену процедуру, переглянути і відредагувати текст існуючої процедури?

Нову процедуру створюють командою CREATE PROCEDURE, після якої вказують ім’я, параметри (за потреби) і тіло процедури:

```
CREATE PROCEDURE IncreaseKyivBonus
    @amount INT = 100
AS
BEGIN
    UPDATE Customers
    SET BONUS_CM = BONUS_CM + @amount
    WHERE CITY_CM = N'Київ';
END;
GO
```

Команда CREATE PROCEDURE має бути першим оператором пакета, тому перед нею в скрипті ставлять GO. Після створення процедуру викликають командою EXEC IncreaseKyivBonus @amount = 100;.

Переглянути текст процедури можна кількома способами:

1) у SSMS – Object Explorer → база даних → Programmability → Stored Procedures → потрібна процедура → Modify (відкривається шаблон ALTER PROCEDURE);

2) командою sp_helptext N'dbo.IncreaseKyivBonus'; – виводить текст окремими рядками;

3) запитом до системного каталогу: SELECT OBJECT_DEFINITION(OBJECT_ID(N'dbo.IncreaseKyivBonus')); або SELECT definition FROM sys.sql_modules WHERE object_id = OBJECT_ID(N'dbo.IncreaseKyivBonus');.

Відредагувати процедуру можна командою ALTER PROCEDURE (повторно вказують увесь текст) або через SSMS. У SQL Server 2016 SP1 і новіших підтримується також CREATE OR ALTER PROCEDURE, яка створює процедуру, якщо її немає, і змінює, якщо вона вже існує. Видалити процедуру можна командою DROP PROCEDURE dbo.IncreaseKyivBonus; або DROP PROCEDURE IF EXISTS dbo.IncreaseKyivBonus;.

4. Опишіть використання параметрів у збережених процедурах.

Параметри дозволяють передавати в процедуру вхідні значення й отримувати результати. Їх оголошують після назви процедури у вигляді @ім’я тип_даних [= значення_за_замовчуванням] [OUTPUT]. Напрямок параметра визначає OUTPUT: без нього параметр є вхідним.

Вхідні параметри можна передавати позиційно або за іменем. Якщо в параметра є значення за замовчуванням, його можна не вказувати. Приклад із цієї роботи:

```
CREATE PROCEDURE ReassignSellerOrders
    @from NVARCHAR(50),
    @to NVARCHAR(50)
AS
BEGIN
    DECLARE @from_id INT, @to_id INT;
    SELECT @from_id = ID_SP FROM Salespeople WHERE NAME_SP = @from;
    SELECT @to_id = ID_SP FROM Salespeople WHERE NAME_SP = @to;
    IF @from_id IS NULL OR @to_id IS NULL
    BEGIN
        RAISERROR(N'Продавця не знайдено', 16, 1);
        RETURN;
    END;
    UPDATE Orders SET ID_SP = @to_id WHERE ID_SP = @from_id;
END;
GO

EXEC ReassignSellerOrders @from = N'Чепуренко', @to = N'Маренич';
```

Вихідний параметр оголошують словом OUTPUT і в виклику також вказують OUTPUT. Фактичним аргументом має бути змінна відповідного типу:

```
CREATE PROCEDURE ReaderLoanReport
    @reader INT,
    @total INT OUTPUT,
    @returned INT OUTPUT
AS
BEGIN
    SELECT @total = COUNT(*),
           @returned = SUM(CASE WHEN return_date IS NOT NULL THEN 1 ELSE 0 END)
    FROM BookLoan
    WHERE id_reader = @reader;
END;
GO

DECLARE @total INT, @returned INT;
EXEC ReaderLoanReport @reader = 1,
     @total = @total OUTPUT, @returned = @returned OUTPUT;
SELECT @total AS Усього_видач, @returned AS Повернуто_книг;
```

Особливості: типи параметрів перевіряються під час виклику; значення за замовчуванням має бути константою; параметри – локальні для процедури й не видні після її завершення; ім’я параметра не повинно збігатися з іменем стовпця, інакше його слід екранувати. Кількість і типи фактичних аргументів мають відповідати оголошенню.

5. Які оператори Transact SQL можна використати при написанні збережених процедур?

У тілі збереженої процедури можна використовувати практично всі оператори Transact-SQL, які допустимі в пакеті:

1) DML: SELECT, INSERT, UPDATE, DELETE, MERGE, TRUNCATE TABLE;

2) DDL: CREATE/ALTER/DROP TABLE, INDEX, VIEW, FUNCTION (з обмеженнями), але не CREATE PROCEDURE всередині іншої процедури;

3) оголошення й присвоєння змінних: DECLARE, SET, SELECT @x = ...;

4) керуючі конструкції: IF…ELSE, WHILE, BREAK, CONTINUE, GOTO, RETURN, BEGIN…END, WAITFOR;

5) обробка помилок: TRY…CATCH, THROW, RAISERROR, PRINT;

6) транзакції: BEGIN TRANSACTION, COMMIT, ROLLBACK, SAVE TRANSACTION;

7) виклики інших процедур і функцій: EXEC, EXEC sp_executesql (динамічний SQL);

8) курсори: DECLARE cursor, OPEN, FETCH, CLOSE, DEALLOCATE;

9) тимчасові таблиці (#temp, ##global) і табличні змінні (@table);

10) агрегатні, рядкові, датові, математичні та системні функції.

Приклад процедури, яка поєднує оголошення змінних, IF, RAISERROR і UPDATE:

```
CREATE PROCEDURE ReturnBook
    @loan INT,
    @date DATE = NULL
AS
BEGIN
    IF NOT EXISTS (SELECT 1 FROM BookLoan WHERE id_loan = @loan)
    BEGIN
        RAISERROR(N'Видачу не знайдено', 16, 1);
        RETURN;
    END;
    IF @date IS NULL SET @date = CAST(GETDATE() AS DATE);
    UPDATE BookLoan SET return_date = @date WHERE id_loan = @loan;
END;
GO
```

Не можна всередині процедури створювати іншу процедуру, а також використовувати деякі команди, які вимагають окремого пакета (наприклад, CREATE VIEW, CREATE TRIGGER, CREATE PROCEDURE). Для них застосовують динамічний SQL або окремі пакети.

6. Як оформляється пакетний файл процедури? Яке призначення завершальної команди пакета GO? Наведіть приклад.

Пакет (batch) – це набір операторів, які клієнтська програма надсилає серверу й сервер виконує як одну логічну одиницю. Межа пакета позначається командою GO. Важливо пам’ятати, що GO – не оператор Transact-SQL, а роздільник пакетів, який розпізнають SSMS, sqlcmd та інші клієнтські інструменти; серверу він не передається.

Команда CREATE PROCEDURE має бути першим (і єдиним) оператором свого пакета. Саме тому перед нею ставлять GO, а після тіла процедури – ще один GO, щоб відокремити створення від подальших викликів:

```
USE Sales111;
GO

CREATE PROCEDURE IncreaseKyivBonus
    @amount INT = 100
AS
BEGIN
    UPDATE Customers
    SET BONUS_CM = BONUS_CM + @amount
    WHERE CITY_CM = N'Київ';
END;
GO

EXEC IncreaseKyivBonus @amount = 100;
GO
```

GO може мати числовий аргумент – кількість повторень пакета (наприклад, GO 5 виконає пакет п’ять разів). Якщо в пакеті сталася синтаксична помилка, він не виконується повністю; змінні, оголошені в одному пакеті, не видні в іншому. Тому в скриптах процедур GO ставлять між логічно самостійними частинами.

7. Опишіть використання конструкції IF… ELSE у збереженій процедурі. Наведіть приклад.

Конструкція IF…ELSE дозволяє виконувати різні оператори залежно від умови. Синтаксис: IF умова оператор1 [ELSE оператор2]. Якщо потрібно виконати кілька операторів, їх об’єднують блоком BEGIN…END. Умова має бути логічним виразом, який повертає TRUE, FALSE або UNKNOWN; для перевірки наявності рядків використовують EXISTS, для NULL – IS NULL.

Приклад із процедури ReturnBook: спочатку перевіряється, чи існує видача, потім – чи вона вже повернута, і лише після цього виконується UPDATE:

```
IF NOT EXISTS (SELECT 1 FROM BookLoan WHERE id_loan = @loan)
BEGIN
    RAISERROR(N'Видачу не знайдено', 16, 1);
    RETURN;
END;

IF EXISTS (SELECT 1 FROM BookLoan
           WHERE id_loan = @loan AND return_date IS NOT NULL)
BEGIN
    RAISERROR(N'Книга вже повернута', 16, 1);
    RETURN;
END
ELSE
BEGIN
    UPDATE BookLoan SET return_date = @date WHERE id_loan = @loan;
END
```

Конструкція IF…ELSE може бути вкладеною; для складних перевірок краще використовувати послідовні IF з RETURN, як у наведеному прикладі, – це робить код читабельнішим. Важливо: IF не є циклом і не підтримує конструкцію ELSE IF – замість неї пишуть вкладений IF або CASE.

8. Опишіть використання конструкції BEGIN… END у збереженій процедурі. Наведіть приклад.

BEGIN…END – це блок операторів, який сприймається як один складений оператор. Його використовують у тілі процедури, після IF…ELSE, у циклах WHILE, у блоках TRY…CATCH. Блоки можна вкладати один в один; вони не створюють транзакцію й не змінюють область видимості змінних.

Приклад – тіло процедури AddBook, де блоки BEGIN…END застосовано і для тіла процедури, і для кожної гілки перевірки:

```
CREATE PROCEDURE AddBook
    @title NVARCHAR(100),
    @author INT,
    @publisher INT,
    @year INT,
    @price MONEY
AS
BEGIN
    IF @price <= 0
    BEGIN
        RAISERROR(N'Ціна книги повинна бути додатною', 16, 1);
        RETURN;
    END;

    IF NOT EXISTS (SELECT 1 FROM Author WHERE id_author = @author)
    BEGIN
        RAISERROR(N'Автора не існує', 16, 1);
        RETURN;
    END;

    INSERT INTO Book (title, id_author, id_publisher, year_pub, price)
    VALUES (@title, @author, @publisher, @year, @price);
END;
GO
```

Якщо після IF має виконуватися лише один оператор, BEGIN…END можна не писати, але для наочності й безпеки під час подальших змін блок ставлять завжди. Незакритий BEGIN – типова синтаксична помилка, тому блоки рекомендується форматувати з відступами.

9. Опишіть використання операторів WHILE, BREAK, CONTINUE. Наведіть приклади.

WHILE організовує цикл: WHILE умова оператор або блок BEGIN…END. Тіло циклу виконується, поки умова істинна. BREAK достроково перериває цикл, CONTINUE переходить до наступної ітерації, пропускаючи решту операторів тіла. Для уникнення нескінченного циклу змінна умови має змінюватися всередині тіла.

Приклад обчислення суми чисел від 1 до 10:

```
DECLARE @i INT = 1, @sum INT = 0;
WHILE @i <= 10
BEGIN
    SET @sum = @sum + @i;
    SET @i = @i + 1;
END;
SELECT @sum AS Сума;
```

Приклад із BREAK і CONTINUE – обробляються лише парні числа, а після 8 цикл переривається:

```
DECLARE @i INT = 0;
WHILE @i < 20
BEGIN
    SET @i = @i + 1;
    IF @i % 2 = 1 CONTINUE;
    IF @i > 8 BREAK;
    PRINT @i;
END;
```

У збережених процедурах WHILE часто застосовують для посторінкової обробки даних, наприклад для видалення великої кількості рядків невеликими порціями, щоб не блокувати таблицю надовго. Для обробки рядків також використовують курсори, але вони повільніші за множинні оператори.

10. Опишіть порядок визначення і використання змінних у збереженій процедурі. Які бувають типи змінних? Наведіть приклади.

Локальні змінні оголошують командою DECLARE @ім’я тип_даних [= початкове_значення];. Значення присвоюють командою SET @x = вираз; або SELECT @x = стовпець FROM ...;. Змінні існують від моменту оголошення до кінця пакета або процедури; їхні імена починаються з @.

Типи змінних:

1) локальні змінні (@x) – оголошуються користувачем;

2) параметри процедури – вхідні та вихідні (OUTPUT);

3) табличні змінні (@t TABLE (...)) – зберігають набір рядків у пам’яті;

4) глобальні системні змінні (@@ROWCOUNT, @@ERROR, @@IDENTITY, @@VERSION) – обчислюються сервером, доступні лише для читання.

Приклад оголошення, присвоєння та використання змінних у процедурі:

```
DECLARE @total INT, @returned INT, @avg MONEY;

SELECT @total = COUNT(*),
       @returned = SUM(CASE WHEN return_date IS NOT NULL THEN 1 ELSE 0 END)
FROM BookLoan
WHERE id_reader = 1;

SELECT @avg = AVG(price) FROM Book;
SELECT @total AS Усього, @returned AS Повернуто, @avg AS Середня_ціна;
```

Особливості: змінна може мати значення NULL; якщо SELECT не повернув рядків, змінна зберігає попереднє значення (а не NULL), тому перед

присвоєнням іноді виконують SET @x = NULL. Оголошувати змінну з тим самим іменем двічі в одному пакеті не можна. Вихідні параметри процедури – це теж змінні, значення яких повертається викликачу.

11. Опишіть використання оператора PRINT. Наведіть приклад.

PRINT надсилає клієнту визначене користувачем повідомлення (до 4000 символів Unicode). Це не результуючий набір: повідомлення з'являється на вкладці Messages у SSMS або в потоці повідомлень клієнта. PRINT зручно використовувати для налагодження та інформування про хід виконання, але не для передачі даних застосунку.

Приклад:

```
DECLARE @cnt INT = (SELECT COUNT(*) FROM Book);
PRINT N'Кількість книг у базі: ' + CAST(@cnt AS NVARCHAR(10));

IF @cnt = 0
    PRINT N'База порожня.';
ELSE
    PRINT N'Дані наявні.';
```

Для формування рядка використовують конкатенацію + і функції CAST або CONVERT, оскільки PRINT приймає лише символьний вираз. Для повідомлень про помилки з встановленням прапора помилки застосовують RAISERROR або THROW, а не PRINT.

12. Опишіть особливості використання глобальних змінних. Наведіть приклад.

Глобальні (системні) змінні – це спеціальні змінні, які обчислює сам сервер; вони починаються з @@ і доступні лише для читання. Їх не оголошують командою DECLARE. Найуживаніші:

1) @@ROWCOUNT – кількість рядків, оброблених останнім оператором;

2) @@ERROR – код помилки останнього оператора (0, якщо помилки не було);

3) @@IDENTITY – останнє значення ідентифікатора, згенероване в поточному з'єднанні;

4) @@TRANCOUNT – кількість активних транзакцій;

5) @@SERVERNAME, @@VERSION, @@SPID – ім’я сервера, версія, ідентифікатор процесу.

Приклад:

```
UPDATE Customers SET BONUS_CM = BONUS_CM + 10 WHERE CITY_CM = N'Київ';
SELECT @@ROWCOUNT AS Оброблено_рядків, @@ERROR AS Код_помилки;

SELECT @@SERVERNAME AS Сервер, @@VERSION AS Версія;
```

Особливість: @@ERROR і @@ROWCOUNT змінюються після кожного оператора, тому їх потрібно зберігати в локальну змінну відразу після перевірюваної команди. У сучасному коді замість @@ERROR частіше використовують TRY…CATCH і функції ERROR_NUMBER(), ERROR_MESSAGE(), ERROR_LINE().

13. Опишіть використання операторів GOTO, RETURN. Наведіть приклади.

GOTO виконує безумовний перехід до мітки в межах пакета або процедури. Мітка – це ідентифікатор, після якого стоїть двокрапка. GOTO застосовують для виходу з кількох вкладених циклів або для переходу до блоку обробки помилки; у структурованому коді його використовують рідко, оскільки він ускладнює читання.

```
DECLARE @i INT = 1;
WHILE @i <= 10
BEGIN
    IF @i = 5 GOTO finish;
    SET @i = @i + 1;
END
finish:
PRINT N'Цикл перервано на значенні ' + CAST(@i AS NVARCHAR(3));
```

RETURN негайно завершує виконання процедури (або пакета) і може повернути ціле значення – код завершення. Код 0 зазвичай означає успіх, ненульове значення – помилку. RETURN не є оператором Transact-SQL для виходу з циклу: для цього призначений BREAK.

```
CREATE PROCEDURE CheckPrice
    @price MONEY
AS
BEGIN
    IF @price <= 0
    BEGIN
        PRINT N'Некоректна ціна';
        RETURN 1;
    END;
    PRINT N'Ціна коректна';
    RETURN 0;
END;
GO

DECLARE @rc INT;
EXEC @rc = CheckPrice @price = 300;
SELECT @rc AS Код_завершення;
```

У процедурах RETURN без аргументу використовують для дострокового виходу після перевірки умов, як у процедурах цієї роботи після RAISERROR.

14. Опишіть використання оператора RAISERROR. Наведіть приклад. Як із збереженої процедури можна повернути визначене користувачем повідомлення про помилку і встановити прапор помилки? Наведіть приклад.

RAISERROR формує повідомлення про помилку й повертає його клієнту. Синтаксис: RAISERROR(msg, severity, state) [WITH NOWAIT]. Параметр msg – текст (до 2047 символів) або номер повідомлення; severity – рівень серйозності від 0 до 25; state – число від 1 до 127, яке допомагає локалізувати місце помилки. Повідомлення з рівнем 11–16 вважаються помилками користувача: вони встановлюють прапор помилки, і клієнт отримує виняток. Рівень 17–19 – критичні помилки, 20–25 – фатальні (з’єднання розривається).

Щоб повернути власне повідомлення, використовують текст із рівнем 16 або власний номер від 50000 і більше (його попередньо додають через sp_addmessage). Приклад із цієї роботи:

```
IF @price <= 0
BEGIN
    RAISERROR(N'Ціна книги повинна бути додатною', 16, 1);
    RETURN;
END;
```

Після виконання такого фрагмента клієнт отримує повідомлення рівня 16, прапор помилки встановлюється, а наступні оператори процедури не

виконуються (завдяки RETURN). Перевірити код останньої помилки можна через @@ERROR або в блоці CATCH:

```
BEGIN TRY
    RAISERROR(N'Помилка перевірки даних', 16, 1);
END TRY
BEGIN CATCH
    SELECT ERROR_NUMBER() AS Код,
           ERROR_MESSAGE() AS Повідомлення,
           ERROR_LINE() AS Рядок;
END CATCH;
```

У нових версіях SQL Server замість RAISERROR часто використовують THROW, який автоматично встановлює прапор помилки й підтримує повторне генерування помилки в CATCH. RAISERROR залишається зручним для дружніх повідомлень українською мовою.

15. Опишіть використання оператора WAITFOR. Наведіть приклад.

WAITFOR зупиняє виконання пакета або процедури на заданий час або до настання вказаного моменту. Синтаксис: WAITFOR DELAY 'hh:mm:ss' – затримка на вказаний інтервал; WAITFOR TIME 'hh:mm:ss' – очікування до вказаного часу доби. Існує також WAITFOR (RECEIVE ...) для роботи з чергами Service Broker.

Приклад затримки на 5 секунд:

```
PRINT N'Початок';
WAITFOR DELAY '00:00:05';
PRINT N'Минуло 5 секунд';
```

Приклад очікування до 18:00:

```
WAITFOR TIME '18:00:00';
SELECT GETDATE() AS Поточний_час;
```

WAITFOR блокує поточне з’єднання, але не блокує інші з’єднання й не споживає ресурси процесора під час очікування. Його використовують для демонстрацій, організації розкладів, повторних спроб із затримкою та тестування конкурентного доступу. Для тривалих очікувань краще застосовувати SQL Server Agent.

16. Опишіть використання оператора CASE. Наведіть приклад.

CASE – це вираз, який повертає одне з кількох значень залежно від умови. Існують дві форми: проста (CASE вираз WHEN значення THEN результат ...) і пошукова (CASE WHEN умова THEN результат ...). CASE можна використовувати в SELECT, SET, WHERE, ORDER BY, а також усередині агрегатних функцій.

Приклад категоризації замовлень за сумою:

```
SELECT ID_OD, AMOUNT_OD,
       CASE
           WHEN AMOUNT_OD >= 5000 THEN N'велике'
           WHEN AMOUNT_OD >= 1000 THEN N'середнє'
           ELSE N'мале'
       END AS Категорія
FROM Orders
ORDER BY AMOUNT_OD DESC;
```

Приклад у процедурі ReaderLoanReport – підрахунок повернутих книг через CASE усередині SUM:

```
SELECT @total = COUNT(*),
       @returned = SUM(CASE WHEN return_date IS NOT NULL THEN 1 ELSE 0 END)
FROM BookLoan
WHERE id_reader = @reader;
```

CASE повертає значення лише одного типу (або сумісного), тому всі гілки THEN мають бути зведені до спільного типу. Якщо жодна умова не виконалася і немає ELSE, результат – NULL. CASE не є керуючою конструкцією: він не виконує оператори, а лише обчислює вираз.

Висновки:

Під час виконання лабораторної роботи № 7 було опановано створення та використання збережених процедур у MS SQL Server. Розглянуто команди CREATE PROCEDURE, ALTER PROCEDURE і DROP PROCEDURE, призначення роздільника пакетів GO, вхідні та вихідні параметри, локальні змінні, конструкції IF…ELSE і BEGIN…END, оператори WHILE, BREAK, CONTINUE, PRINT, RAISERROR, RETURN, GOTO, WAITFOR і CASE. Для бази даних Sales111 створено вісім процедур: ChangeOrderDate, DeleteCustomerOrders, IncreaseKyivBonus, ReassignSellerOrders, DeleteCustomersWithoutOrders, DeleteSellersWithoutOrders, IncreaseTopSellerCommission і LongestCustomerName. На їх прикладі показано зміну дати замовлень, видалення рядків, збільшення бонусів і комісійних, перепризначення замовлень і пошук найдовшого прізвища. Для бази даних LibraryDB (варіант № 1 «Бібліотека») створено три процедури – AddBook, ReturnBook і ReaderLoanReport, – які демонструють INSERT, UPDATE, перевірку даних через IF…ELSE і RAISERROR та повернення результатів через вихідні параметри OUTPUT. Усі процедури перевірено виконанням; змінені під час демонстрації дані відновлено до початкового стану. Надано відповіді на всі шістнадцять контрольних питань.